Multi-Server Java Application Build, Store, and Deployment System

Objective

The objective of this task is to set up a multi-server environment for building, storing, and deploying a Java web application using a Build Server, Nexus Repository Server, and Deploy Server.

Servers Used

1. Build Server – Build Java application using Maven
2. Nexus Repository Server – Store WAR artifacts
3. Deploy Server – Deploy application on Apache Tomcat

1. Java Build Server Setup

Step 1: Connect to Build Server

ssh -i "devops.pem" ubuntu@ec2-3-91-216-228.compute-1.amazonaws.com

Step 2: Install Java and Maven

sudo apt update

sudo apt install openjdk-17-jdk -y

sudo apt install maven -y

java -version

mvn -version

Step 3: Clone GitHub Repository
git clone https://github.com/mrtechreddy/Java-Web-Calculator-App.git
cd Java-Web-Calculator-App

```
ubuntu@Build-server:~$ ls
Java-Web-Calculator-App  devops.pem
ubuntu@Build-server:~$
```

Step 4: Build Application
mvn clean package

```
[INFO]
[INFO] --- war:3.3.2:war (default-war) @ webapp-add ---
[INFO] Packaging webapp
[INFO] Assembling webapp [webapp-add] in [/home/ubuntu/Java-Web-Calculator-App/target/webapp-add-0.0.3]
[INFO] Processing war project
[INFO] Copying webapp resources [/home/ubuntu/Java-Web-Calculator-App/src/main/webapp]
[INFO] Building war: /home/ubuntu/Java-Web-Calculator-App/target/webapp-add-0.0.3.war
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  3.614 s
[INFO] Finished at: 2026-05-17T10:00:48Z
[INFO] ------------------------------------------------------------------------
```

WAR file generated inside target/ directory.

```
ubuntu@Build-server:~$ cd Java-Web-Calculator-App/
ubuntu@Build-server:~/Java-Web-Calculator-App$ ls
Jenkinsfile  README.md  pom.xml  src  target
ubuntu@Build-server:~/Java-Web-Calculator-App$ cd target/
ubuntu@Build-server:~/Java-Web-Calculator-App/target$ ls
classes            generated-test-sources  maven-status     test-classes      webapp-add-0.0.3.war
generated-sources  maven-archiver          surefire-reports webapp-add-0.0.3
ubuntu@Build-server:~/Java-Web-Calculator-App/target$
```

Step 5: Configure pom.xml
Add Nexus repository configuration under distributionManagement.

```
 <distributionManagement>
   <repository>
     <id>maven-releases</id>
     <url>http://3.80.155.124:8081/repository/maven-releases/</url>
   </repository>
 </distributionManagement>

</project>
```

Step 6: Configure Maven settings.xml
Add Nexus credentials in /etc/maven/settings.xml.

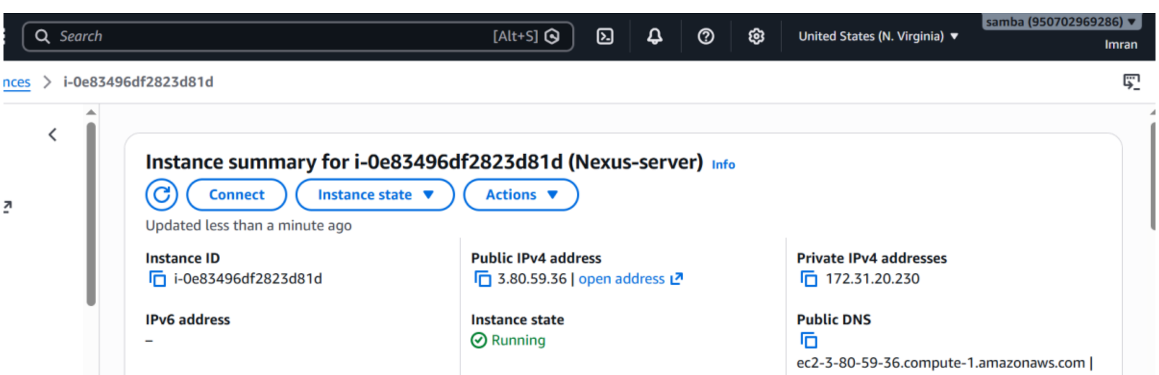
```
    <!-- Nexus Authentication -->
<servers>

        <server>
<id>maven-releases</id>
<username>admin</username>
<password>admin123</password>
</server>

        <server>
<id>maven-snapshots</id>
<username>admin</username>
<password>admin</password>
</server>

    </servers>

    <!-- Mirrors -->
<mirrors>
```

Step 7: Deploy Artifact to Nexus
mvn deploy

```
[INFO]
[INFO] --- war:3.3.2:war (default-war) @ webapp-add ---
[INFO] Packaging webapp
[INFO] Assembling webapp [webapp-add] in [/home/ubuntu/Java-Web-Calculator-App/target/webapp-add-0.0.3]
[INFO] Processing war project
[INFO] Copying webapp resources [/home/ubuntu/Java-Web-Calculator-App/src/main/webapp]
[INFO] Building war: /home/ubuntu/Java-Web-Calculator-App/target/webapp-add-0.0.3.war
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  3.614 s
[INFO] Finished at: 2026-05-17T10:00:48Z
```

2. Nexus Repository Server Setup

Step 1: Install Java
sudo apt update
sudo apt install openjdk-17-jdk -y



```
ubuntu@Nexus-server:~$ java --version
openjdk 17.0.18 2026-01-20
OpenJDK Runtime Environment (build 17.0.18+8-Ubuntu-1)
OpenJDK 64-Bit Server VM (build 17.0.18+8-Ubuntu-1, mixed mode, sharing)
ubuntu@Nexus-server:~$
```

Step 2: Download Nexus

```
14  wget https://download.sonatype.com/nexus/3/nexus-3.85.0-03-linux-x86_64.tar.gz
```

```
ubuntu@Nexus-server:~$ ls
nexus-3.85.0-03  nexus-3.85.0-03-linux-x86_64.tar.gz  sonatype-work
ubuntu@Nexus-server:~$
```

Step 3: Extract Nexus

tar -xvf latest-unix.tar.gz

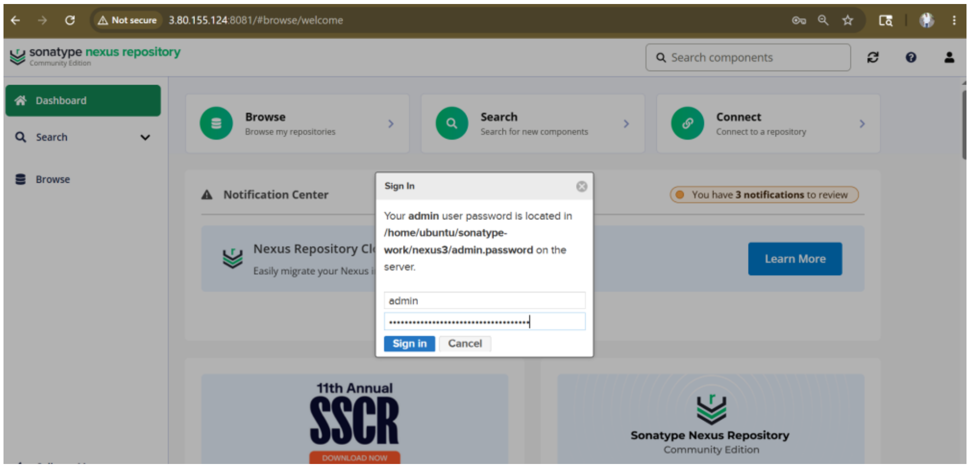
Start Nexus

cd /opt/nexus/bin

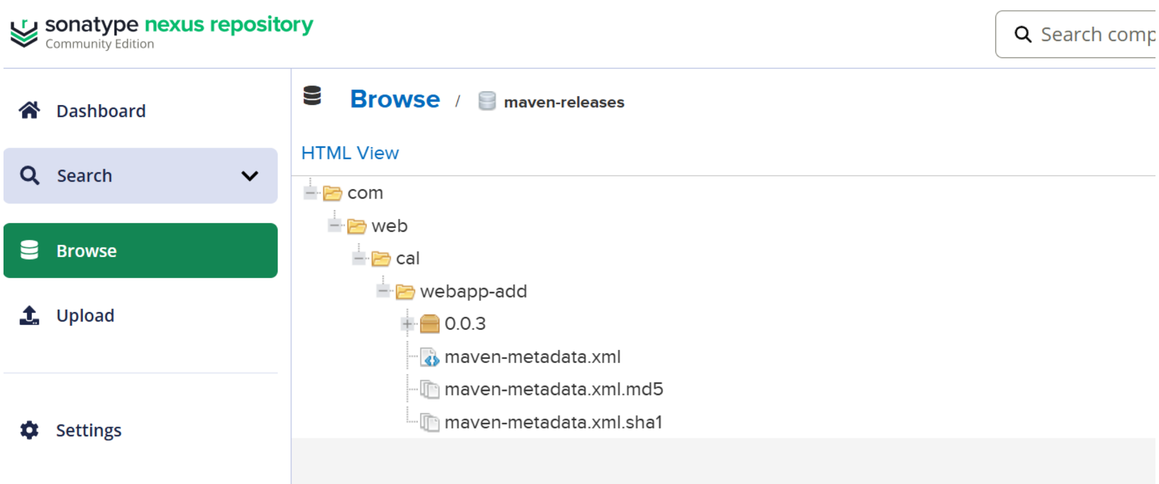
./nexus start

```
16  tar -xvf nexus-3.85.0-03-linux-x86_64.tar.gz
17  ls
18  cd nexus-3.85.0-03/
19  ls
20  cd bin
21  ls
22  ./nexus
23  ./nexus start
24  ./nexus statuss
25  ./nexus status
```

Step 5: Access Nexus Web Interface
http://3.80.59.36:8081



Step 6: Create Maven Hosted Repository
Create repository named maven-releases.

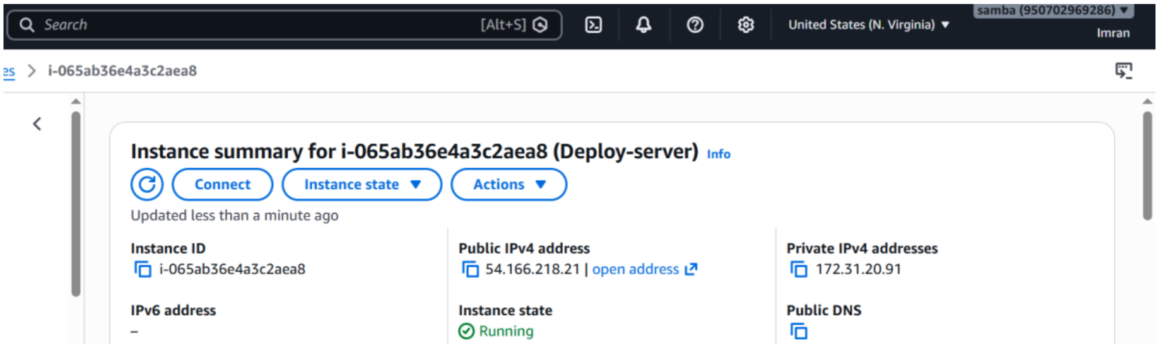


3. Deploy Server Setup

Step 1: Install Java and Apache Tomcat

sudo apt update

sudo apt install openjdk-17-jdk -y

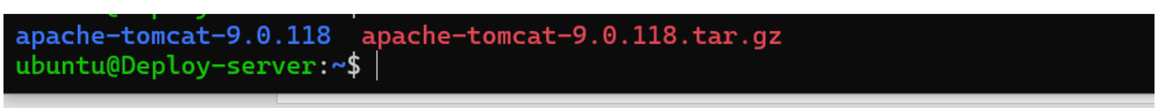


Download Tomcat:

wget https://downloads.apache.org/tomcat/tomcat-9/v9.0.118/bin/apache-tomcat-9.0.118.tar.gz

Extract Tomcat:

tar -xvf apache-tomcat-9.0.118.tar.gz

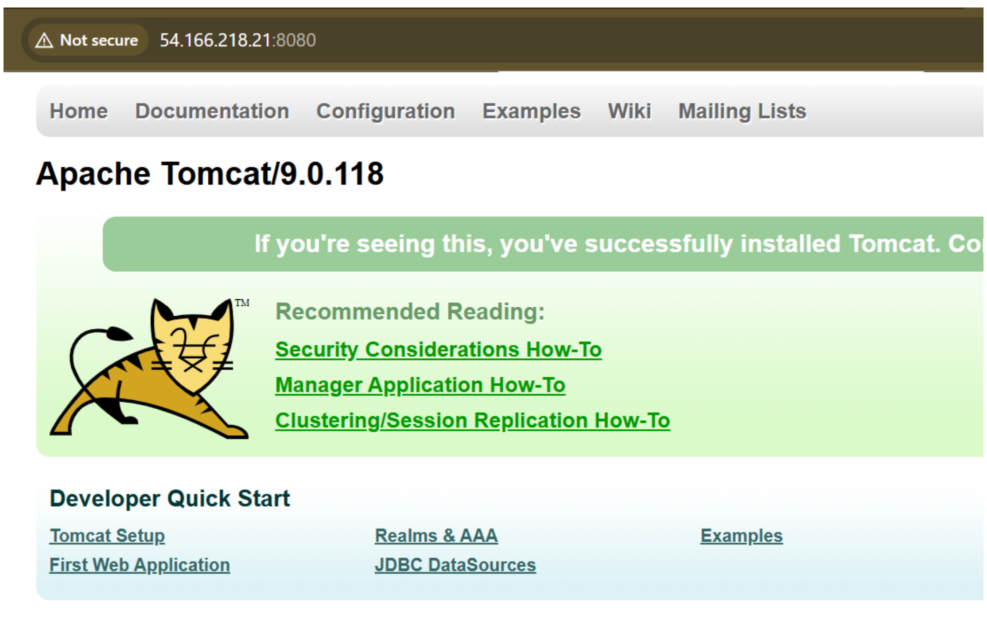


Step 2: Start Tomcat

cd /opt/tomcat/bin

./startup.sh

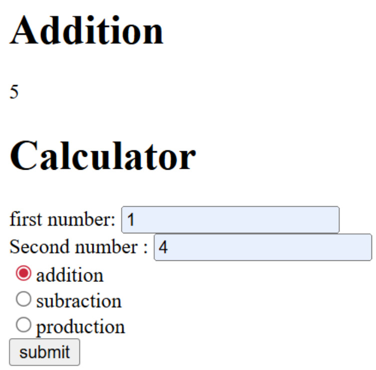
```
ubuntu@Deploy-server:~$ cd apache-tomcat-9.0.118/
ubuntu@Deploy-server:~/apache-tomcat-9.0.118$ ls
BUILDING.txt     LICENSE  README.md      RUNNING.txt  conf  logs  webapps
CONTRIBUTING.md  NOTICE   RELEASE-NOTES  bin          lib   temp  work
ubuntu@Deploy-server:~/apache-tomcat-9.0.118$ cd bin
ubuntu@Deploy-server:~/apache-tomcat-9.0.118/bin$ ls
bootstrap.jar       ciphers.sh                   daemon.sh     setclasspath.bat  startup.sh           version.
catalina-tasks.xml  commons-daemon-native.tar.gz digest.bat    setclasspath.sh   tomcat-juli.jar      version.
catalina.bat        commons-daemon.jar           digest.sh     shutdown.bat      tomcat-native.tar.gz
catalina.sh         configtest.bat               makebase.bat  shutdown.sh       tool-wrapper.bat
ciphers.bat         configtest.sh                makebase.sh   startup.bat       tool-wrapper.sh
ubuntu@Deploy-server:~/apache-tomcat-9.0.118/bin$ ./startup.sh
Using CATALINA_BASE:   /home/ubuntu/apache-tomcat-9.0.118
Using CATALINA_HOME:   /home/ubuntu/apache-tomcat-9.0.118
Using CATALINA_TMPDIR: /home/ubuntu/apache-tomcat-9.0.118/temp
Using JRE_HOME:        /usr
Using CLASSPATH:       /home/ubuntu/apache-tomcat-9.0.118/bin/bootstrap.jar:/home/ubuntu/apache-tomcat-9.0.118/b
t-juli.jar
Using CATALINA_OPTS:
Tomcat started.
```



Step 3: Copy WAR File to Deploy Server

```
126  scp -i /home/ubuntu/devops.pem Java-Web-Calculator-App/target/webapp-add-0.0.3.war ubuntu@54.160.139.225:/home/ub
ntu/apache-tomcat-9.0.118/webapps/
```

Step 4: Access Application
http://54.166.218.21:8080



Issues Encountered and Fixes

1. Permission denied for PEM file → Fixed using chmod 400 devops.pem
2. Maven deployment failed → Corrected Nexus repository ID and credentials
3. Tomcat startup issue → Verified Java installation and Tomcat logs
4. SCP connection issue → Updated security groups and PEM permissions

Conclusion

Successfully implemented a complete multi-server Java application deployment pipeline using Apache Maven, Nexus Repository Manager, and Apache Tomcat.